1775c019-589d-4e74-8571-f9966f05463b

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'camunda/camunda' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/camunda/camunda (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-10T14:09:25.029Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add sample outputs for test collection script
</pr_title>

<pr_description>
Description

Provides concrete examples of the test collection script's output, serving as baseline documentation of the current test landscape (3,551 tests across the monorepo).

Sample outputs added:
- `output-text.txt` - All tests in plain text format (471 KB)
- `output-json.json` - Complete JSON output (711 KB)
- `output-json-sample.json` - First 50 tests for quick review

(11 KB)
- `collection-stderr.log` - Progress messages and warnings
- `metrics.txt` - Summary statistics

Key findings:
- 3,551 total tests: 76% unit, 13% integration, 8% acceptance
- 86 unclassified tests requiring investigation
- 41 utility classes detected (enhanced detection of Spring Boot test fixtures)
- Top team: @camunda/core-features with 1,717 tests

Enhanced utility class detection:
- Expanded heuristic to detect Spring Boot test fixtures like `TestApiController`, `TestUserDetailsService`
- New regex pattern catches `Test*Controller`, `Test*Service`, `Test*Repository`, `Test*Config`, etc.
- Now detects 41 utility classes (up from 17 initially)

Sample directory includes README with regeneration instructions.

Checklist

-] Enable backports when necessary (fex. [for bug fixes, for CI changes, or for documentation changes).

Related issues

N/A - Documentation and tooling improvement

<issue_title>Implement script to collect all tests (file | type | owner)</issue_title>
<   → "acceptance"
   │
2. QA MODULE DETECTION (surefire skipped, failsafe runs everything)
   ├── Path under qa/*/ (not acceptance/archunit)   → "integration"
   ├── Path under zeebe/qa/

```
*/                                          → "integration"
   ├── Path under operate/qa/
*/                                        → "integration"
   ├── Path under tasklist/qa/
*/                                      → "integration"
   │   Note: even *Test.java files here — they ALL run via
failsafe (double check this!)
   │
3. SUFFIX-BASED (standard Maven defaults — only for non-qa
modules)
   ├── *IT.java, IT*.java,
*ITCase.java                                → "integration"
   ├── *Test.java, Test*.java, *Tests.java,
*TestCase.java          → "unit"
   │
5. FALLBACK
   └── Any remaining test-like
file                                    → "other, output a warning"
```

Rules are evaluated top-to-bottom; first match wins - also think about how this would evolve in the future or what happens when someone adds a test using wrong naming/placement (would we still detect that via a verify step using maven as source of truth)

Verification safeguard: `--verify` mode

The script includes a `--verify` flag that cross-checks its heuristic classification against Maven's actual plugin resolution. This is the **authoritative source of truth** and should be run periodically to catch classification drift.

How `--verify` works

1. **Discover all Maven modules**: parse `pom.xml` `<modules>` recursively
2. **For each module with test sources**: run `mvn -pl <module> surefire:test failsafe:integration-test -X -DskipTests` and parse debug output for `[DEBUG] Selecting test class: <fqcn>` lines
3. **Compare**: surefire-selected → expect `unit`, failsafe-selected → expect `integration`/`acceptance`/etc.
4. **Report mismatches**: output a diff showing disagreements between heuristic and Maven

Make sure this doesn't actually run the tests as it would take to long, it should just trigger a dry run to explore all tests that WOULD run.

When to run
- After adding new modules or restructuring `qa/` directories
- After adding new tests to ensure they will be captured in CI jobs
- After changing surefire/failsafe plugin config in any `pom.xml`
- As a periodic CI check (optional — can be expensive for full monorepo)

Limitations
- Slow (runs Maven per module) — not for every invocation
- Profile-specific test configurations may not be captured in default run
</issue_description>

Comments on the Issue (you are @claude[agent] in this section)

<comments>
</comments>

</pr_description>

Comments on the PR (you are @claude[agent] in this section)

<comments>

<pr_comments>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>scripts/test-collection/collect-tests.js:3</file>
<original_commit>5b8f363</original_commit>

</comment_thread>

<comment_thread>
<file>scripts/test-collection/collect-tests.js:56</file>
<original_commit>5b8f363</original_commit>

</comment_thread>

<comment_thread>
<file>scripts/test-collection/README.md:14</file>
<original_commit>5b8f363</original_commit>

</comment_thread>

<comment_thread>
<file>scripts/test-collection/README.md:102</file>
<original_commit>5b8f363</original_commit>

</comment_thread>

<comment_thread>
<file>scripts/test-collection/README.md:130</file>
<original_commit>5b8f363</original_commit>

</comment_thread>

<comment_thread>
<file>scripts/test-collection/test-classification.js:4</file>
<original_commit>5b8f363</original_commit>

</comment_thread>

</pr_reviews>

<pr_comments>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>scripts/codeowners-utils/samples/collection-stderr.log:129</file>
<original_commit>a5b1878</original_commit>

</comment_thread>

<comment_thread>
<file>scripts/codeowners-utils/samples/README.md:63</file>
<original_commit>a5b1878</original_commit>

```
<comment_new>
<comment_id>2911970105</comment_id>
<author>@eppdot</author>
@claude[agent] can you still run the maven across a few modules and provide the output like requested?
</comment_new>
```

</comment_thread>

<comment_thread>
<file>scripts/codeowners-utils/README.md:139</file>
<original_commit>a5b1878</original_commit>

```
<comment_new>
<comment_id>2911978396</comment_id>
<author>@eppdot</author>
@claude[agent] do we show a "diff" between both strategies
```

```
(heuristic and maven)?
</comment_new>
```

</comment_thread>

</pr_reviews>

</comments>

The last **6** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Based on my analysis:

- The **backport_template.md** is specifically for backporting existing

PRs to other branches, which is not applicable here.
- The **pull_request_template.md** is the default/general template for all PRs.

This PR is adding documentation and tooling improvements (sample outputs for the test collection script). It's not a backport, so the default template is the appropriate choice.

<template_path>/home/runner/work/camunda/camunda/.github/pull_request_template.md</template_path>
<template_content>
Description

<!-- Describe the goal and purpose of this PR. -->

Checklist

<!--- Please delete options that are not relevant. Boxes should be checked by reviewer. -->
- [ ] Enable backports when necessary (fex. [for bug fixes](https://github.com/camunda/camunda/blob/main/CONTRIBUTING.md#backporting-changes), [for CI changes](https://camunda.github.io/camunda/ci/#when-to-backport-ci-changes), or [for documentation changes](https://camunda.github.io/camunda/ci/#documentation-specific-backporting-monorepo-docs-folders)).

Related issues

closes #
</template_content>

<current>

<request>

You are working on an issue in the 'camunda/camunda' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/camunda/camunda (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-10T14:41:29.020Z</current_datetime>

Consider the following problem statement:

<problem_statement>
Have you run the maven strategy in your environment/session? is it really working? thus when I checkout the code, how confident are you that it runs without errors on my machine?
</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.